

本节内容

顺序表

定义

知识总览

线性表

逻辑结构

基本运算/操作

存储/物理结构



顺序表 (顺序存储)

链表 (链式存储)

定义 (如何用代码实现)

基本操作的实现

顺序表的定义



顺序表——用顺序存储的方式实现线性表顺序存储。把逻辑上相邻的元素存储在物理位置上也相邻的存储单元中，元素之间的关系由存储单元的邻接关系来体现。

```
typedef struct {  
    int num; //号数  
    int people; //人数  
} Customer;
```

如何知道一个数据元素大小?
C语言 **sizeof(ElemType)**
Eg:

sizeof(int) = 4B
sizeof(Customer) = 8B



顺序表的实现——静态分配

```
#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int length;
}SqList;
```

Sq: sequence —— 顺序，序列

//定义最大长度

//用静态的“数组”存放数据元素

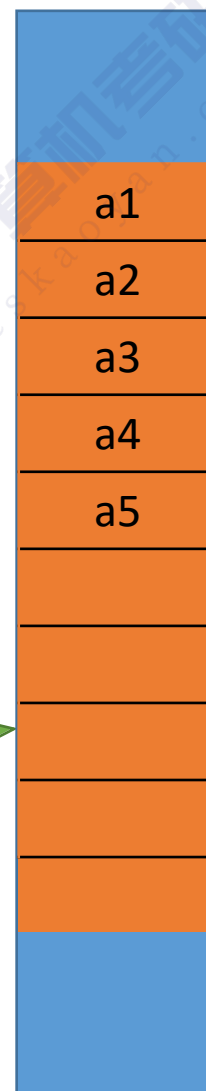
//顺序表的当前长度

//顺序表的类型定义（静态分配方式）



给各个数据元素分配连续的存储空间，大小为 $\text{MaxSize} * \text{sizeof}(\text{ElemType})$

内存



```
#include <stdio.h>
#define MaxSize 10 //定义最大长度
typedef struct{
    int data[MaxSize]; //用静态的“数组”存放数据元素
    int length; //顺序表的当前长度
}SqList; //顺序表的类型定义
```

本例中数据元素的类型（ElemType）是 int

//基本操作—初始化一个顺序表

```
void InitList(SqList &L){
```

② → for(int i=0; i<MaxSize; i++)

L.data[i]=0; //将所有数据元素设置为默认初始值

③ → L.length=0; //顺序表初始长度为0

```
}
```

```
int main() {
```

① → SqList L; //声明一个顺序表

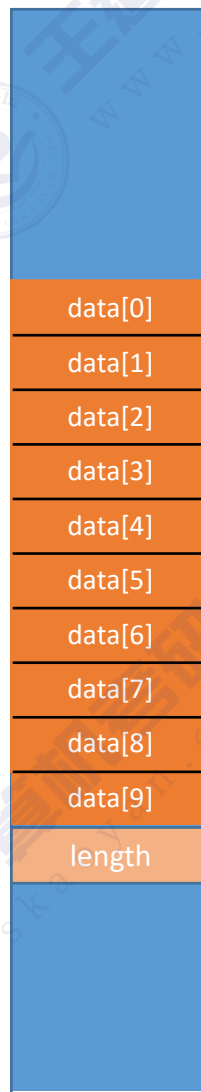
→ InitList(L); //初始化顺序表

//.....未完待续，后续操作

```
return 0;
```

```
}
```

内存



②把各个数据元素的值设为默认值（可省略）

①在内存中分配存储顺序表L的空间。包括：
MaxSize*sizeof(ElemType)
和 存储 length 的空间

③将 Length 的值设为0

```

/*不初始化数据元素，内存不刷0*/
#include <stdio.h>
#define MaxSize 10 //定义最大长度
typedef struct{
    int data[MaxSize]; //用静态的“数组”存放数据元素
    int length; //顺序表的当前长度
}SqList; //顺序表的类型定义

```

```

//基本操作—初始化一个顺序表
void InitList(SqList &L){
    ③ L.length=0; //顺序表初始长度为0
}

```

```

① int main() {
    SqList L; //声明一个顺序表
    InitList(L); //初始化顺序表
    //尝试“违规”打印整个 data 数组
    for(int i=0; i<MaxSize; i++)
        printf("data[%d]=%d\n", i, L.data[i]);
    return 0;
}

```

`i < L.length;`

没有设置数据元素的默认值

内存中会有遗留的“脏数据”

这种访问方式也不够好，更好的做法是使用基本操作来访问各个数据元素

内存



②把各个数据元素的值设为默认值（可省略）

①在内存中分配存储顺序表L的空间。包括：
MaxSize*sizeof(ElemType) 和 存储 length 的空间

③将 Length 的值设为0

思考：这一步是否可省略？

```

data[0]=0
data[1]=0
data[2]=0
data[3]=0
data[4]=0
data[5]=0
data[6]=0
data[7]=0
data[8]=-272632568
data[9]=32766

```

顺序表的实现——静态分配

```
#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int length;
}SqList;
```

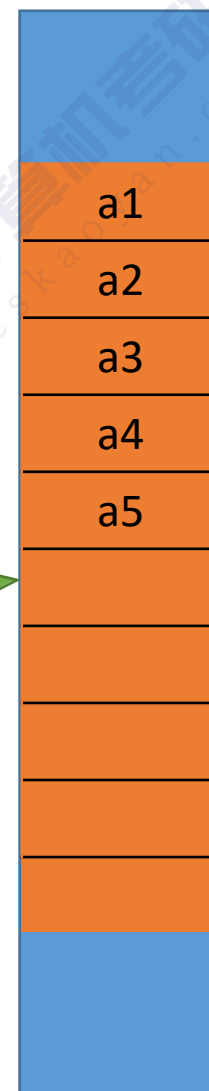
//定义最大长度

//用静态的“数组”存放数据元素
//顺序表的当前长度
//顺序表的类型定义



给各个数据元素分配连续的存储空间，大小为
 $\text{MaxSize} * \text{sizeof}(\text{ElemType})$

内存



Q: 如果“数组”存满了怎么办?

A: 可以放弃治疗，顺序表的表长刚开始确定后就无法更改（存储空间是静态的）

思考：如果刚开始就声明一个很大的内存空间呢？存在什么问题？



同学，浪费是不行的

王道考研/CSKAOYAN.COM

顺序表的实现——动态分配

```
#define InitSize 10 //顺序表的初始长度
typedef struct{
    ElemType *data; //指示动态分配数组的指针
    int MaxSize; //顺序表的最大容量
    int length; //顺序表的当前长度
} SeqList; //顺序表的类型定义（动态分配方式）
```

Key: 动态申请和释放内存空间

C —— malloc、free 函数

```
L.data = (ElemType *) malloc (sizeof(ElemType) * InitSize);
```

C++ —— new、delete 关键字

malloc 函数返回一个指针，
需要强制转型为你定义的数据元素类型指针

malloc 函数的参数，指明要分配多大的连续内存空间

内存



malloc、free
函数的头文件

```
#include <stdlib.h>
```

```
#define InitSize 10 //默认的最大长度
typedef struct{
    int *data; //指示动态分配数组的指针
    int MaxSize; //顺序表的最大容量
    int length; //顺序表的当前长度
}SeqList;
```

```
int main() {
    SeqList L; //声明一个顺序表
    InitList(L); //初始化顺序表
    //...往顺序表中随便插入几个元素...
    IncreaseSize(L, 5);
    return 0;
}
```

注：realloc 函数也可实现，但建议初学者使用 malloc 和 free 更能理解背后过程

顺序表的实现——动态分配

```
void InitList(SeqList &L){
    //用 malloc 函数申请一片连续的存储空间
    L.data=(int *)malloc(InitSize*sizeof(int));
    L.length=0;
    L.MaxSize=InitSize;
}
```

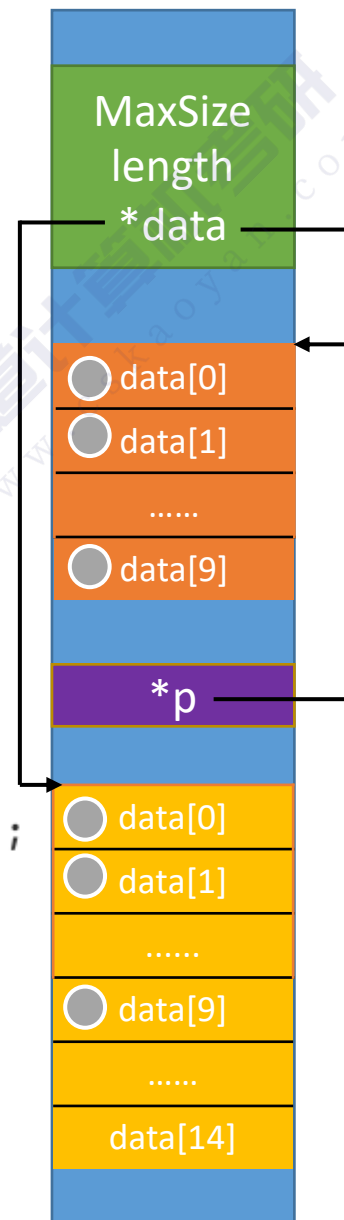
```
//增加动态数组的长度
void IncreaseSize(SeqList &L, int len){
    int *p=L.data;
    L.data=(int *)malloc((L.MaxSize+len)*sizeof(int));
    for(int i=0; i<L.length; i++){
        L.data[i]=p[i];
    }
    L.MaxSize=L.MaxSize+len;
    free(p);
}
```

//将数据复制到新区域

//顺序表最大长度增加 len
//释放原来的内存空间

时间开销大

内存

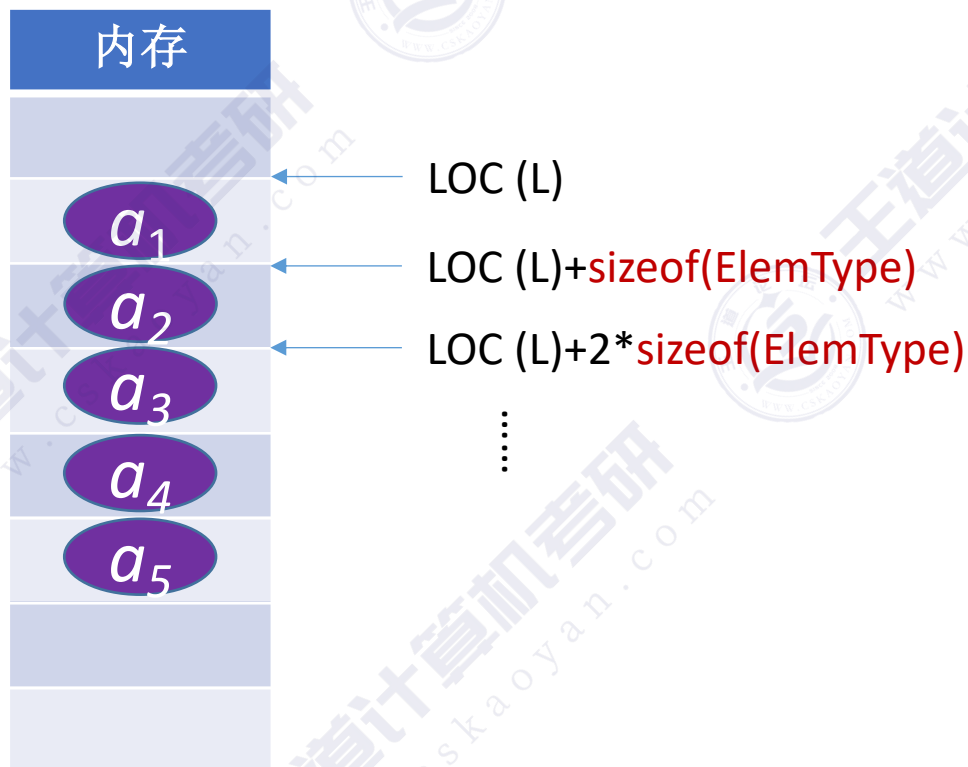


顺序表的实现

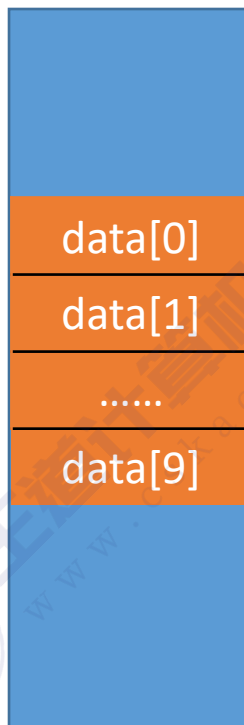
顺序表的特点:

- ① **随机访问**, 即可以在 $O(1)$ 时间内找到第 i 个元素。
- ② 存储密度高, 每个节点只存储数据元素
- ③ 拓展容量不方便 (即便采用动态分配的方式实现, 拓展长度的时间复杂度也比较高)
- ④ 插入、删除操作不方便, 需要移动大量元素

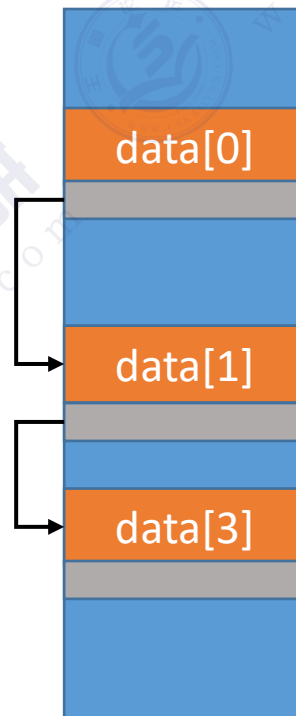
代码实现: `data[i-1];`
静态分配、动态分配都一样



顺序存储



链式存储



存放指针

知识回顾与重要考点

顺序表

存储结构

逻辑上相邻的数据元素物理上也相邻

实现方式

静态分配

使用“静态数组”实现

大小一旦确定就无法改变

动态分配

使用“动态数组”实现

`L.data = (ElemType *) malloc (sizeof(ElemType) * size);`

顺序表存满时，可再用 `malloc` 动态拓展顺序表的最大容量

需要将数据元素复制到新的存储区域，并用 `free` 函数释放原区域

特点

随机访问

能在 $O(1)$ 时间内找到第 i 个元素

存储密度高

拓展容量不方便

插入、删除数据元素不方便

理解原理

动手写一写
`malloc`、`free`

本节内容

顺序表

插入
删除

知识总览

顺序表的插入和删除

插入

代码实现

时间复杂度分析

删除

代码实现

时间复杂度分析

用存储位置的相邻来体现数据元素之间的逻辑关系

顺序表的基本操作——插入

位序

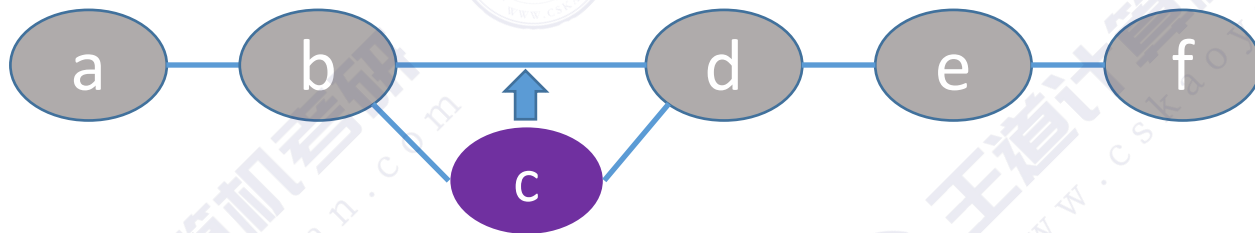
ListInsert(&L,i,e): 插入操作。在表L中的第i个位置上插入指定元素e。

```
#define MaxSize 10 //定义最大长度
typedef struct{
    ElemType data[MaxSize]; //用静态的“数组”存放数据元素
    int length; //顺序表的当前长度
}SqList; //顺序表的类型定义
```

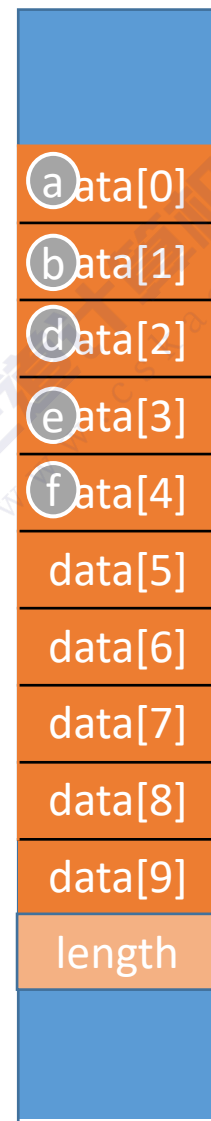
用静态分配方式实现的顺序表

注：本节代码建立在顺序表的“静态分配”实现方式之上，“动态分配”也雷同。

逻辑结构:



内存



顺序表的基本操作——插入

```
#define MaxSize 10 //定义最大长度
typedef struct{
    int data[MaxSize]; //用静态的“数组”存放数据元素
    int length; //顺序表的当前长度
}SqList; //顺序表的类型定义
```

```
void ListInsert(SqList &L, int i, int e){
    for(int j=L.length; j>=i; j--) //将第i个元素及之后的元素后移
        L.data[j]=L.data[j-1];
    L.data[i-1]=e; //在位置i处放入e
    L.length++; //长度加1
}
```

```
int main() {
    SqList L; //声明一个顺序表
    InitList(L); //初始化顺序表
    //...此处省略一些代码，插入几个元素
    ListInsert(L, 3, 3);
    return 0;
}
```

基本操作：在L的位序i处插入元素e

注意位序、数组下标的关系，并从后面的元素依次移动

ListInsert(L, 9, 3);

基操——让自己实现的数据结构可以让别人很方便地使用

我可能就是天才吧

内存



= 6

顺序表的基本操作——插入

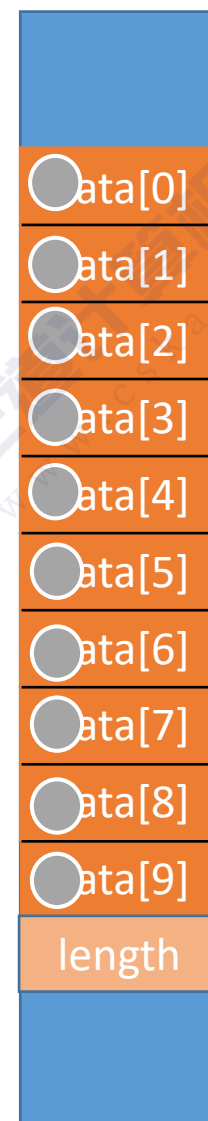
```
#define MaxSize 10          //定义最大长度
typedef struct{
    int data[MaxSize]; //用静态的“数组”存放数据元素
    int length;         //顺序表的当前长度
}SqList;                  //顺序表的类型定义

bool ListInsert(SqList &L, int i, int e){
    if(i<1||i>L.length+1) //判断i的范围是否有效
        return false;
    if(L.length>=MaxSize) //当前存储空间已满，不能插入
        return false;
    for(int j=L.length;j>=i;j--) //将第i个元素及之后的元素后移
        L.data[j]=L.data[j-1];
    L.data[i-1]=e; //在位置i处放入e
    L.length++; //长度加1
    return true;
}
```



好的算法，应该具有“健壮性”。
能处理异常情况，并给使用者反馈

内存



插入操作的时间复杂度

```
bool ListInsert(SqList &L, int i, int e){  
    if(i<1||i>L.length+1)           //判断i的范围是否有效  
        return false;  
    if(L.length>=MaxSize)             //当前存储空间已满, 不能插入  
        return false;  
    for(int j=L.length;j>=i;j--)      //将第i个元素及之后的元素后移  
        L.data[j]=L.data[j-1];  
    L.data[i-1]=e;                     //  
    L.length++;                        //长度加1  
    return true;  
}
```

关注最深层循环语句的执行次数与问题规模 n 的关系

问题规模 $n = L.length$ (表长)

最好情况: 新元素插入到表尾, 不需要移动元素

$i = n+1$, 循环0次; 最好时间复杂度 = $O(1)$

最坏情况: 新元素插入到表头, 需要将原有的 n 个元素全都向后移动

$i = 1$, 循环 n 次; 最坏时间复杂度 = $O(n)$;

平均情况: 假设新元素插入到任何一个位置的概率相同, 即 $i = 1, 2, 3, \dots, length+1$ 的概率都是 $p = \frac{1}{n+1}$

$i = 1$, 循环 n 次; $i = 2$ 时, 循环 $n-1$ 次; $i = 3$, 循环 $n-2$ 次 $i = n+1$ 时, 循环0次

平均循环次数 = $np + (n-1)p + (n-2)p + \dots + 1 \cdot p = \frac{n(n+1)}{2} \cdot \frac{1}{n+1} = \frac{n}{2}$ → 平均时间复杂度 = $O(n)$

A diagram of a linked list with six nodes labeled a, b, c, d, e, and f. The nodes are represented as ovals connected by a horizontal line. Node c is highlighted in purple, and a blue arrow points down from it, indicating a deletion operation.

a data[0]
b data[1]
c data[2]
d data[3]
e data[4]
f data[5]
data[6]
data[7]
data[8]
data[9]
length



顺序表的基本操作——删除

内存

① data[0]

② data[1]

③ data[2]

④ data[3]

⑤ data[4]

⑥ data[5]

data[6]

data[7]

data[8]

data[9]

length

= 6

③ e

复制

```
bool ListDelete(SqList &L, int i, int &e){  
    if(i<1||i>L.length) //判断i的范围是否有效  
        return false;  
    e=L.data[i-1]; //将被删除的元素赋值给e  
    for(int j=i;j<L.length;j++) //将第i个位置后的元素前移  
        L.data[j-1]=L.data[j];  
    L.length--; //线性表长度减1  
    return true;  
}
```

注意位序、数组下标的关系，并从前面的元素依次移动

```
int main() {  
    SqList L; //声明一个顺序表  
    InitList(L); //初始化顺序表  
    //...此处省略一些代码，插入几个元素  
    int e = -1; //用变量e把删除的元素“带回来”  
    if (ListDelete(L, 3, e))  
        printf("已删除第3个元素,删除元素值为%d\n", e);  
    else  
        printf("位序i不合法,删除失败\n");  
    return 0;  
}
```

如果参数没有加引用符号，会怎样？



思考考

已删除第3个元素,删除元素值为=3
Program ended with exit code: 0

删除操作的时间复杂度

```
bool ListDelete(SqList &L, int i, int &e){  
    if(i<1||i>L.length)           //判断i的范围是否有效  
        return false;  
    e=L.data[i-1];                 //将被删除的元素赋值给e  
    for(int j=i;j<L.length;j++)    //将第i个位置后的元素前移  
        L.data[j-1]=L.data[j];  
    L.length--;  
    return true;  
}
```

关注最深层循环语句的执行
次数与问题规模 n 的关系

问题规模 $n = L.length$ (表长)

最好情况：删除表尾元素，不需要移动其他元素

$i = n$ ，循环 0 次；最好时间复杂度 = $O(1)$

最坏情况：删除表头元素，需要将后续的 $n-1$ 个元素全都向前移动

$i = 1$ ，循环 $n-1$ 次；最坏时间复杂度 = $O(n)$;

平均情况：假设删除任何一个元素的概率相同，即 $i = 1, 2, 3, \dots, length$ 的概率都是 $p = \frac{1}{n}$

$i = 1$ ，循环 $n-1$ 次； $i = 2$ 时，循环 $n-2$ 次； $i = 3$ ，循环 $n-3$ 次 $i = n$ 时，循环 0 次

$$\text{平均循环次数} = (n-1)p + (n-2)p + \dots + 1 \cdot p = \frac{n(n-1)}{2} \cdot \frac{1}{n} = \frac{n-1}{2}$$

➡ 平均时间复杂度 = $O(n)$

知识回顾与重要考点

顺序表的基本操作

插入

ListInsert(&L,i,e)

将元素 e 插入到 L 的第 i 个位置

插入位置之后的元素都要后移

时间复杂度

最好 $O(1)$ 、最坏 $O(n)$ 、平均 $O(n)$

删除

ListDelete(&L,i,&e)

将 L 的第 i 个元素删除，并用 e 返回

删除位置之后的元素都要前移

时间复杂度

最好 $O(1)$ 、最坏 $O(n)$ 、平均 $O(n)$

代码要点

代码中注意位序 i 和数组下标的区别

算法要有健壮性，注意判断 i 的合法性

跨考同学注意：移动元素时，从靠前的元素开始？还是从表尾元素开始？

分析代码，理解为什么有的参数需要加“&”引用

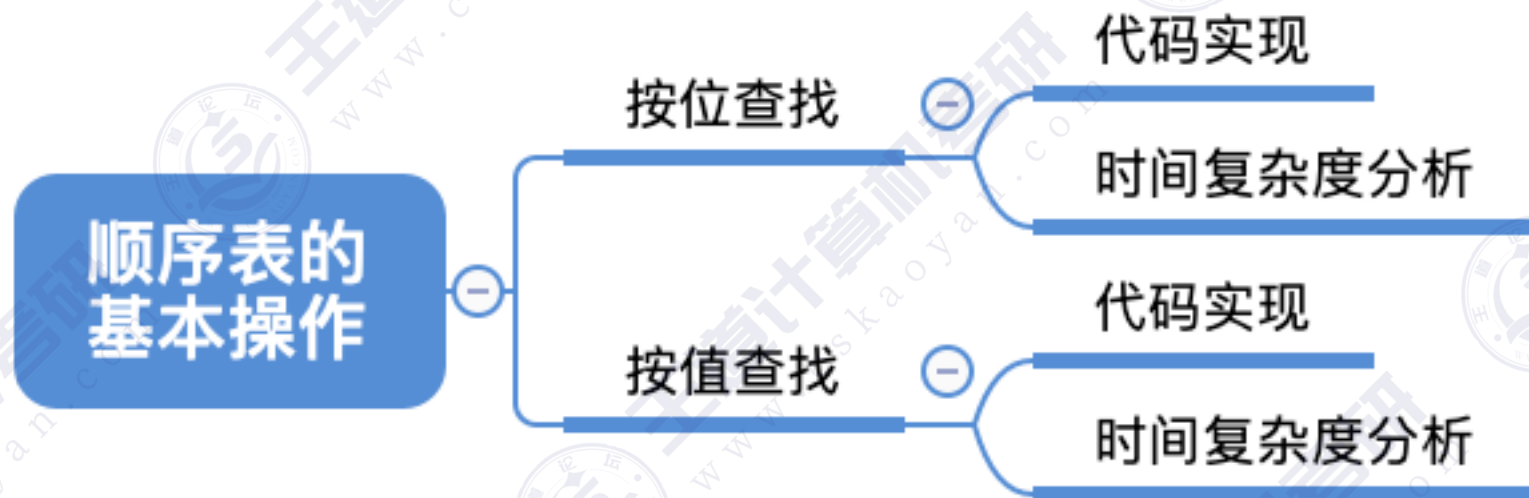
如果不加“&”，则被调用函数中处理的是参数数据的复制品

本节内容

顺序表

查找

知识总览



顺序表的按位查找

GetElem(L,i): 按位查找操作。获取表L中第i个位置的元素的值。

```
#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int length;
}SqList;
```

//定义最大长度

//用静态的“数组”存放数据元素
//顺序表的当前长度
//顺序表的类型定义（静态分配方式）

静态分配

```
ElemType GetElem(SqList L, int i){
    return L.data[i-1];
}
```

顺序表的按位查找

GetElem(L,i): 按位查找操作。获取表L中第i个位置的元素的值。

```
#define InitSize 10
```

```
typedef struct{
```

```
    ElemType *data;
```

```
    int MaxSize;
```

```
    int length;
```

```
} SeqList;
```

//顺序表的初始长度

//指示动态分配数组的指针

动态分配

//顺序表的最大容量

//顺序表的当前长度

//顺序表的类型定义（动态分配方式）

```
ElemType GetElem(SeqList L, int i){
```

```
    return L.data[i-1];
```

```
}
```

和访问普通数组的方法一样

```
ElemType *data
```

如果一个 ElemType 占 6B，即 sizeof(ElemType)==6
指针 data 指向的地址为 2000

内存



data[0]

data[1]

data[2]

.....

(从2000开始的6B) (从2006开始的6B) (从2012开始的6B)

顺序表的按位查找

GetElem(L,i): 按位查找操作。获取表L中第i个位置的元素的值。

```
#define InitSize 10
typedef struct{
    ElemType *data;
    int MaxSize;
    int length;
} SeqList;
```

//顺序表的初始长度

//指示动态分配数组的指针

动态分配

//顺序表的最大容量

//顺序表的当前长度

//顺序表的类型定义（动态分配方式）

```
ElemType GetElem(SeqList L, int i){
    return L.data[i-1];
}
```

和访问普通数组的方法一样

如果换一个类型的指针，指向同一个地址

int *p

p[0]

p[1]

p[2]

p[4]

..... (注：一个int 占4B)

内存



data[0]

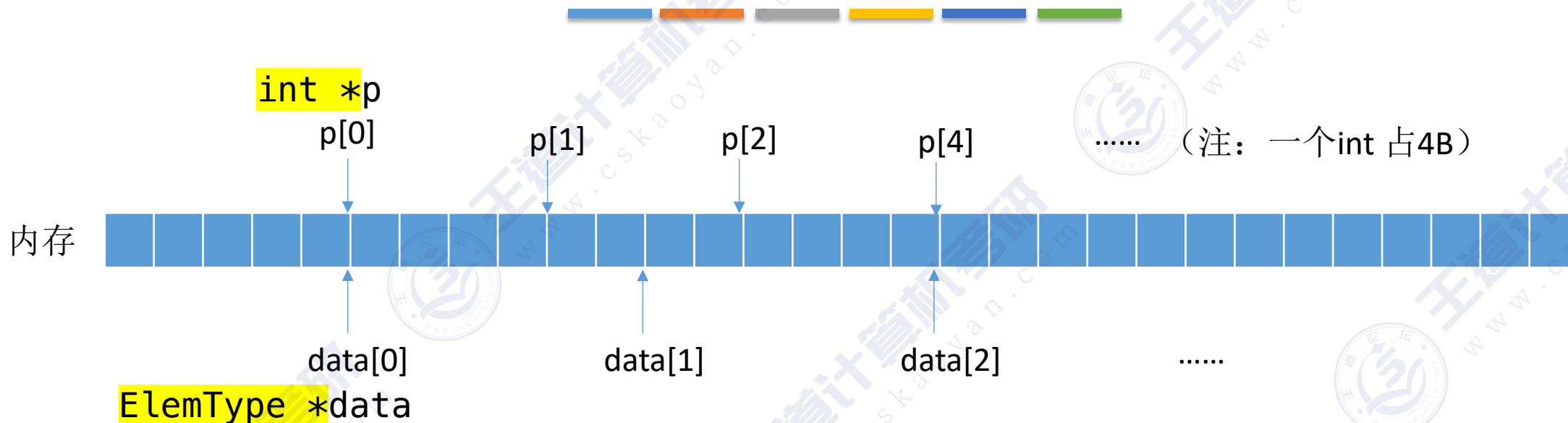
data[1]

data[2]

.....

(从2000开始的6B) (从2006开始的6B) (从2012开始的6B)

顺序表的按位查找



```
#define InitSize 10 //默认的最大长度
typedef struct{
    int *data; //指示动态分配数组的指针
    int MaxSize; //顺序表的最大容量
    int length; //顺序表的当前长度
}SeqList;
```

```
void InitList(SeqList &L){
    //用 malloc 函数申请一片连续的存储空间
    L.data=(int *)malloc(InitSize*sizeof(int));
    L.length=0;
    L.MaxSize=InitSize;
}
```

再次理解，为何malloc函数返回的存储空间起始地址要转换为与数据元素的数据类型相对应的指针

按位查找的时间复杂度

内存



ElemType *data

```
ElemType GetElem(SeqList L, int i){  
    return L.data[i-1];  
}
```

时间复杂度: $O(1)$

由于顺序表的各个数据元素在内存中连续存放，因此可以根据起始地址和数据元素大小立即找到第*i*个元素——“随机存取”特性

顺序表的按值查找

LocateElem(L,e): 按值查找操作。在表L中查找具有给定关键字值的元素。

```
#define InitSize 10           //顺序表的初始长度
typedef struct{               //指示动态分配数组的指针
    ElemType *data;           //顺序表的最大容量
    int MaxSize;              //顺序表的当前长度
    int length;               //顺序表的类型定义（动态分配方式）
} SeqList;

//在顺序表L中查找第一个元素值等于e的元素，并返回其位序
int LocateElem(SeqList L,ElemType e){
    for(int i=0;i<L.length;i++)
        if(L.data[i]==e)
            return i+1;       //数组下标为i的元素值等于e，返回其位序i+1
    return 0;                 //退出循环，说明查找失败
}
```

顺序表的按值查找

LocateElem(L,e): 按值查找操作。在表L中查找具有给定关键字值的元素。

```
typedef struct{  
    int *data;           //指示动态分配数组的指针  
    int MaxSize;         //顺序表的最大容量  
    int length;          //顺序表的当前长度  
}SeqList;
```

//在顺序表L中查找第一个元素值等于e的元素，并返回其位序

```
int LocateElem(SeqList L,int e){  
    for(int i=0;i<L.length;i++){  
        if(L.data[i]==e){  
            return i+1;  
        }  
    }  
    return 0;  
}
```

调用: LocateElem(L, 9);

内存

i=0

i=1

i=2

data[0]

data[1]

data[2]

data[3]

data[4]

data[5]

data[6]

data[7]

data[8]

data[9]

length

L.data

= 6

基本数据类型: int、char、double、float 等可以直接用运算符“==”比较

结构类型的数据元素也这样吗?

结构类型的比较

```
typedef struct {  
    int num;  
    int people;  
} Customer;
```

```
void test () {  
    Customer a;  
    a.num = 1;  
    a.people = 1;  
    Customer b;  
    b.num = 1;  
    b.people = 1;  
    if (a == b) {  
        printf("相等");  
    }else {  
        printf("不相等");  
    }  
}
```



不能

```
bool isCustomerEqual (Customer a, Customer b){  
    if (a.num == b.num && a.people == b.people)  
        return true;  
    else  
        return false;  
}
```

更好的办法：定义一个函数，童叟无欺，用过都说好

注意：C语言中，结构体的比较不能直接用“==”

需要依次对比各个分量来判断两个结构体是否相等

```
if (a.num == b.num && a.people == b.people) {  
    printf("相等");  
}else {  
    printf("不相等");  
}
```

顺序表的按值查找

LocateElem(L,e): 按值查找操作。在表L中查找具有给定关键字值的元素。

//在顺序表L中查找第一个元素值等于e的元素，并返回其位序

```
int LocateElem(SeqList L, ElemType e){  
    for(int i=0; i<L.length; i++)  
        if(L.data[i]==e)  
            return i+1;    //数组下标为i的元素值等于e，返回其位序i+1  
    return 0;              //退出循环，说明查找失败  
}
```

Tips:

《数据结构》考研初试中，手写代码可以直接用“==”，无论 ElemType 是基本数据类型还是结构类型

手写代码主要考察学生是否能理解算法思想，不会严格要求代码完全可运行

有的学校考《C语言程序设计》，那么...也许就要语法严格一些

按值查找的时间复杂度

//在顺序表L中查找第一个元素值等于e的元素，并返回其位序

```
int LocateElem(SeqList L, ElemType e){  
    for(int i=0; i<L.length; i++)  
        if(L.data[i]==e)  
            return i+1;  
    return 0;  
}
```

关注最深层循环语句的执行
次数与问题规模 n 的关系

返回其位序 $i+1$

问题规模 $n = L.length$ (表长)

最好情况：目标元素在表头

循环1次；最好时间复杂度 = $O(1)$

最坏情况：目标元素在表尾

循环 n 次；最坏时间复杂度 = $O(n)$;

平均情况：假设目标元素出现在任何一个位置的概率相同，都是 $\frac{1}{n}$

目标元素在第1位，循环1次；在第2位，循环2次；.....；在第 n 位，循环 n 次

平均循环次数 = $1 \cdot \frac{1}{n} + 2 \cdot \frac{1}{n} + 3 \cdot \frac{1}{n} + \dots + n \cdot \frac{1}{n} = \frac{n(n+1)}{2} \cdot \frac{1}{n} = \frac{n+1}{2}$ $\Rightarrow$ 平均时间复杂度 = $O(n)$

知识回顾与重要考点

顺序表的基本操作

按位查找

GetElem(L,i)

获取表L中第*i*个位置的元素的值

用数组下标即可得到第*i*个元素 $L.data[i-1]$

时间复杂度

最好/最坏/平均时间复杂度都是 $O(1)$

按值查找

LocateElem(L,e)

在顺序表L中查找第一个元素值等于*e*的元素，并返回其位序

从第一个元素开始依次往后检索

时间复杂度

最好 $O(1)$: 目标元素在第一个位置

最坏 $O(n)$: 目标元素在最后一个位置

平均 $O(n)$: 目标元素在每个位置的概率相同